

```
> bank %>% select(age, job, education)
  age      job education
1   30 unemployed primary
2   33  services secondary
3   35 management tertiary
4   30 management tertiary
5   59 blue-collar secondary
6   35 management tertiary.
```

filter

The *filter()* function is used to produce a subset of the data that matches the input conditions. The bank entries with 'blue-collar' jobs alone can be listed as follows:

```
> bank %>% filter(job == "blue-collar")
  age      job marital education default balance
housing loan  contact
1  59 blue-collar married secondary no 0 yes no unknown
2  31 blue-collar married secondary no 360 yes yes cellular
3  25 blue-collar single primary no -221 yes no unknown
4  55 blue-collar married primary no 627 yes no unknown
5  32 blue-collar married secondary no 2089 yes no cellular
...
```

arrange

The data from selected columns can be sorted using the *arrange()* function. In the following example, the bank data is first sorted by 'age' and then by the 'balance' column:

```
> bank %>% arrange(age, balance)
  age job marital education default balance housing loan
1   19 student single unknown no 0 no no
2   19 student single primary no 103 no no
3   19 student single secondary no 302 no no
4   19 student single unknown no 1169 no no
5   20 student single secondary no 291 no no
6   20 student single secondary no 502 no no
7   20 student single secondary no 1191 no no
8   21 student single secondary no 6 no no
```

relocate

The *relocate()* function is used to change the column positions in the output. You can use the '.before' and '.after' arguments to specify the location of the columns. For example:

```
> bank %>% relocate(age, .after = "job")
  job age marital education default balance housing loan
1 unemployed 30 married primary no 1787 no no
2 services 33 married secondary no 4789 yes yes
3 management 35 single tertiary no 1350 yes no
4 management 30 married tertiary no 1476 yes yes
5 blue-collar 59 married secondary no 0 yes no
```

```
6 management 35 single tertiary no 747 no no
...
```

count

The unique values for a column can be computed using the *count()* function, as shown below:

```
> bank %>% count(age)
  age  n
1  19  4
2  20  3
3  21  7
4  22  9
5  23 20
6  24 24
7  25 44
8  26 77
9  27 94
```

tally

The *tally()* method is a low-level function that can also be used to count unique values for a column. The total number of bank entries in the CSV file is 4521, as indicated below:

```
> bank %>% tally()
  n
1 4521
```

distinct

The unique rows in a data frame can be obtained using the *distinct()* function. The three possible values for marital status are shown below:

```
> bank %>% distinct marital
  marital
1 married
2 single
3 divorced
```

mutate

The *mutate()* function is used to create new columns from existing data. The balance field is in Euros, and a new USD column is computed based on the Euro-USD conversion rate as follows:

```
> bank %>% select(age, education, balance) %>% mutate(usd =
balance * 1.08)
  age education balance usd
1   30 primary 1787 1929.96
2   33 secondary 4789 5172.12
3   35 tertiary 1350 1458.00
4   30 tertiary 1476 1594.08
```